Resumen Maestro: Lectura 6 — Árboles AVL, Balance y Rotaciones

> **Curso:** CC232 - Algoritmos y Estructuras de Datos (UNI)
> **Cátedra:** Prof. César Lara Avila
> **Semana:** 06 — Árboles AVL y Operaciones Auto-Balanceadas
> **Archivo Original:** Lectura6.md

---

1. La Cadena Retrospectiva y Motivación de Ingeniería

```
problema -> BST degenerado O(n) -> campo height -> factor balance -> rotaciones O(1) -> cota O(log n)
```

En las Semanas 4 y 5 construimos y manipulamos un BST simple, demostrando que búsqueda, inserción y eliminación cuestan $O(h)$, donde h es la altura del árbol. Sin embargo, quedó una vulnerabilidad grave sin resolver: **un BST simple no controla su forma**.

Si insertamos las claves ordenadas `10, 20, 30, 40, 50, 60, 70`, el BST resultante es una cadena degenerada donde $h = n = 7$. Las operaciones se degradan a tiempo lineal $O(n)$, perdiendo toda ventaja algorítmica.

El **Árbol AVL (Adelson-Velsky y Landis)** no sustituye al BST: **es un BST con disciplina de altura**. No cambia el significado básico del conjunto ordenado ni la lógica de búsqueda (`izquierda < nodo < derecha`). Lo que cambia radicalmente es la representación y el conjunto de invariantes: **ahora cada nodo almacenará información sobre su altura, y exigiremos una relación entre la altura de sus dos subárboles. Cuando una inserción rompa esa relación, utilizaremos modificaciones locales llamadas rotaciones ($O(1)$)**, forzando matemáticamente que la altura global sea siempre acotada por $h = O(\log n)$.

---

2. La Quíntupla Estructural Canónica

1. **ADT:** Diccionario dinámico ordenado con garantías logarítmicas en el peor caso.
2. **Representación:** Árbol binario enlazado con metadato explícito:

```
static class Node {
    int x;
    int height = 1; // Hoja recién nacida tiene altura 1
    Node left;
    Node right;
    Node(int x) { this.x = x; }
}
```

   - **Reglas Directas de Altura:**
     - `height(null) = 0`: Si la referencia es vacía, no aporta altura.
     - `height(hoja) = 1`: Una hoja recién creada mide 1 nivel (`int height = 1;`).

- Para todo nodo u: $u.\text{height} = 1 + \max(\text{height}(u.\text{left}), \text{height}(u.\text{right}))$.

Métodos Auxiliares de Altura y Balance:

```
private int height(Node u) {
    return u == null ? 0 : u.height;
}

private void updateHeight(Node u) {
    u.height = 1 + Math.max(height(u.left), height(u.right));
}

private int balanceFactor(Node u) {
    return u == null ? 0 : height(u.left) - height(u.right);
}
```

3. **Invariantes Simultáneos Obligatorios:**
   - *Invariante BST:* Subárbol izquierdo $< u.x <$ subárbol derecho (recorrido inorder estrictamente creciente).
   - *Invariante de Altura Almacenada:* En todo nodo, el campo `height` coincide exactamente con la longitud del camino descendente más largo.
   - *Invariante AVL (Factor de Balance):* Para **todo** nodo alcanzable u:

$$\text{balanceFactor}(u) = \text{height}(u.\text{left}) - \text{height}(u.\text{right})$$

$$\text{balanceFactor}(u) \in \{-1, 0, 1\}$$

```
_(Convención de signo: izquierda menos derecha. Positivo significa cargado a la izquierda; negativo,
```

4. **Mecanismos de Reparación:** Rotaciones simples (`rotateRight`, `rotateLeft`) y dobles (LR, RL) en $O(1)$.
5. **Complejidad Asintótica:** Búsqueda e Inserción estrictamente en $O(\log n)$ en el peor caso.

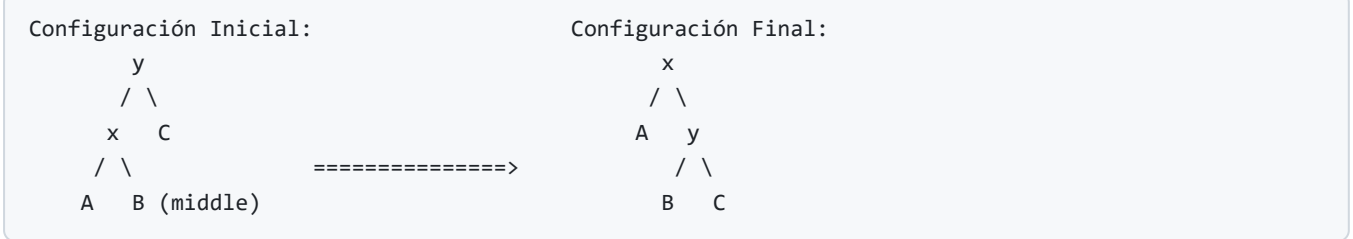
---

3. Anatomía de las Rotaciones (Transformaciones Locales $O(1)$)

Una rotación cambia la topología física del árbol **sin alterar en absoluto el orden inorder**. Modifica 3 referencias y recalcula 2 alturas.

A. Rotación Derecha (`rotateRight(Node y)`): Caso LL

Se aplica cuando el subárbol izquierdo de y creció demasiado. El nodo x sube a ser la nueva raíz local e y baja a su derecha.



La relación inorder se preserva idéntica: $A < x < B < y < C$.

```
static Node rotateRight(Node y) {
    Node x = y.left;
    Node middle = x.right; // 1. Guardar el subárbol intermedio

    // 2. Reconexión local
    x.right = y;
    y.left = middle;

    // 3. Mantenimiento de alturas (¡ORDEN CRÍTICO!)
    updateHeight(y); // Primero el que bajó (ahora es hijo)
    updateHeight(x); // Luego la nueva raíz local (padre)

    return x; // Retornar la nueva raíz local para reasignación
}
```

> **IMPORTANTE**
>
> **¿Por qué `middle` se engancha en `y.left`?** Porque las claves de `middle` son mayores que x y menores que y ($x < \text{middle} < y$). Al convertirse y en hijo derecho de x, el espacio que queda libre para claves menores que y es precisamente `y.left`.

B. Rotación Izquierda (`rotateLeft(Node x)`): Caso RR

Es el reflejo simétrico exacto de la rotación derecha. y sube a ser raíz local y x desciende a su izquierda:

```
static Node rotateLeft(Node x) {
    Node y = x.right;
    Node middle = y.left; // Guardar subárbol intermedio

    y.left = x;
    x.right = middle;

    updateHeight(x); // Primero el que bajó
    updateHeight(y); // Luego la nueva raíz

    return y;
}
```

---

4. Los 4 Patrones Canónicos de Desbalance

Cuando insertamos una clave en un nodo u, revisamos dos signos: el de $\text{balanceFactor}(u)$ (que valdrá $+2$ o -2) y el de su hijo relevante.

```
                        ¿balanceFactor(u)?
                        /               \
          bf(u) = +2                      bf(u) = -2
       (Izquierda Pesada)              (Derecha Pesada)
          /        \                      /        \
bf(u.left) >= 0  bf(u.left) < 0  bf(u.right) <= 0  bf(u.right) > 0
    [LL]             [LR]             [RR]             [RL]
 1 rotación       2 rotaciones     1 rotación       2 rotaciones
```

Tabla Resumen de Diagnóstico y Reparación:

Caso	Diagnóstico Formal	Disposición Geométrica	Reparación Exacta
LL	$\text{bf}(u) > 1 \ \wedge \ \text{bf}(u.\text{left}) \geq 0$	Línea recta a la izquierda (`/`)	`rotateRight(u)`
RR	$\text{bf}(u) < -1 \ \wedge$ $\text{bf}(u.\text{right}) \leq 0$	Línea recta a la derecha (`\`)	`rotateLeft(u)`
LR	$\text{bf}(u) > 1 \ \wedge \ \text{bf}(u.\text{left}) < 0$	Zig-zag izquierda-derecha (`<`)	**1.** `u.left = rotateLeft(u.left)` (alinea a LL) **2.** `return rotateRight(u)`
RL	$\text{bf}(u) < -1 \ \wedge$ $\text{bf}(u.\text{right}) > 0$	Zig-zag derecha-izquierda (`>`)	**1.** `u.right = rotateRight(u.right)` (alinea a RR) **2.** `return rotateLeft(u)`

---

5. Algoritmo Maestro de Inserción Recursiva

La inserción desciende como un BST tradicional y repara de **abajo hacia arriba (postorden)** en la pila de recursión:

```
public void add(int x) {
    root = add(root, x);
}

private Node add(Node u, int x) {
    // 1. Caso Base: Inserción de nueva hoja
    if (u == null) {
        return new Node(x); // height = 1 por defecto
    }

    // 2. Descenso BST
    if (x < u.x) {
        u.left = add(u.left, x);
    } else if (x > u.x) {
        u.right = add(u.right, x);
    } else {
        return u; // Clave duplicada: no se admiten repetidos
    }

    // 3. Viaje de regreso: Actualizar altura del nodo actual
    updateHeight(u);

    // 4. Diagnosticar balance y reparar si es necesario
    return rebalance(u);
}

private Node rebalance(Node u) {
    int bf = balanceFactor(u);

    // Desbalance hacia la izquierda (+2)
    if (bf > 1) {
        if (balanceFactor(u.left) < 0) {
            u.left = rotateLeft(u.left); // Paso 1 del caso LR
        }
        return rotateRight(u); // Caso LL o paso 2 de LR
    }

    // Desbalance hacia la derecha (-2)
    if (bf < -1) {
        if (balanceFactor(u.right) > 0) {
            u.right = rotateRight(u.right); // Paso 1 del caso RL
        }
        return rotateLeft(u); // Caso RR o paso 2 de RL
    }

    // Si bf está en {-1, 0, 1}, el nodo ya es válido
    return u;
}

// Búsqueda en AVL: hereda la lógica BST pero garantizada en O(log n)
public boolean contains(int x) {
    Node w = root;
    while (w != null) {
        if (x < w.x) {
            w = w.left;
        } else if (x > w.x) {
```

```
            w = w.right;
        } else {
            return true;
        }
    }
    return false;
}

// Recorrido de verificación docente: imprime clave y factor de balance
void preorder() {
    preorder(root);
    System.out.println();
}

private void preorder(Node u) {
    if (u == null) return;
    System.out.print(u.x + "(fb=" + balanceFactor(u) + ") ");
    preorder(u.left);
    preorder(u.right);
}
```

6. ¿Por qué la Altura de un AVL es Estrictamente $O(\log n)$?

Un AVL no obliga a que el árbol sea perfectamente simétrico ni lleno. La restricción de que los subárboles hermanos difieran en altura a lo más en 1 nivel impone una cota inferior al número de nodos necesarios para alcanzar una altura h.

El árbol AVL más "flaco" (con la mínima cantidad de nodos $N(h)$ para una altura h) sigue la relación de recurrencia de tipo Fibonacci:

$$N(h) = 1 + N(h-1) + N(h-2)$$

Como esta recurrencia crece a la tasa de la proporción áurea $\phi \approx 1.618$, se demuestra matemáticamente que:

$$h \leq 1.44 \log_2(n+2)$$

Por consiguiente, la altura h está acotada superiormente por $O(\log n)$. Como la búsqueda y la inserción recorren a lo más un camino de longitud h, y cada rotación es $O(1)$, **ambas operaciones quedan garantizadas en $O(\log n)$ en el peor caso**.

7. Contrastes Conceptuales Clave

A. BST Simple frente a Árbol AVL

- **BST Simple:** Solo cuida el orden ($O(h)$ sin cotas). Su representación es barata (sin metadatos), pero vulnerable a entradas ordenadas ($h = O(n)$).
- **Árbol AVL:** Cuida orden y forma. Paga un costo de almacenamiento (`int height` en cada nodo) y sobrecarga en inserción para garantizar $h = O(\log n)$ siempre.

B. Transformación Local frente a Garantía Global

Una rotación solo toca 2 nodos y reasigna 3 punteros ($O(1)$). Sin embargo, esa intervención local restaura el invariante de balance en cascada hacia la raíz, asegurando la propiedad global de altura de toda la estructura.

C. Recorrido Inorder frente a Estado AVL

Un árbol con punteros de altura corruptos (`height = 999`) puede producir un recorrido inorder perfectamente ordenado. El orden de las claves **no prueba** que las alturas ni los factores de balance sean correctos.

---

8. Trampas Fijas de Examen (Catálogo de Errores UNI)

1. **Perder el subárbol intermedio `middle`:** Escribir directamente `x.right = y` sin antes haber respaldado `middle = x.right;`. El subárbol B queda huérfano y se borra de memoria.
2. **Invertir el orden de `updateHeight` tras rotar:** Hacer `updateHeight(x); updateHeight(y);` dentro de `rotateRight`. La nueva raíz x calculará su altura basándose en la altura **antigua** de y, dejando el metadato corrupto.
3. **Olvidar reasignar el hijo al regresar de la recursión:** Escribir `add(u.left, x);` en vez de `u.left = add(u.left, x);`. Si en el nivel inferior ocurrió una rotación que cambió la raíz de ese subárbol, el padre se quedará apuntando al nodo antiguo.
4. **Confundir balanceado con simétrico:** Un nodo con factor $+1$ o -1 **es completamente válido según AVL**. No debe forzarse ninguna rotación si $\text{bf} \in \{-1, 0, 1\}$.
5. **Creer que `add` en AVL es $O(1)$ porque las rotaciones son $O(1)$:** Las rotaciones son $O(1)$, pero la inserción debe descender un camino de longitud $O(\log n)$. Por lo tanto, `add` cuesta $O(\log n)$.

---

9. Comprueba tu Comprensión (Pausa de Trazado)

1. **Traza Numérica:** Inserta en un AVL vacío la secuencia: `50, 30, 70, 20, 10`. Al insertar 10, ¿qué nodo queda desbalanceado, cuál es su balanceFactor, qué caso identifica y cuál es el resultado tras rotar?
2. **Subárbol no vacío:** En `rotateRight(50)` donde $x = 30$, supón que 30 tiene como hijo derecho a 40, y 40 tiene como hijo izquierdo a 35 (`middle = 40(izq 35)`). Dibuja cómo queda enganchado ese subárbol al finalizar la rotación.
3. **Caso Doble:** Si un nodo tiene $\text{bf} = -2$ y su hijo derecho tiene $\text{bf} = +1$, ¿qué caso es, qué rotación previa se aplica sobre el hijo y cuál sobre el nodo padre?

> *Verifica estas tres preguntas trazando cajas y flechas en papel antes de continuar.*